LISTA DE ESTUDOS DE CASO • TRILHA START

Estudos de caso: lógica de programação

*JavaScript para resolver em sala • Aulas 1 a 10*Prof. Douglas Ribeiro

Aluno(a): _____

Turma: _____ **Data:** ____/____/_____

Como usar esta lista

Os casos seguem a ordem das aulas. Cada um parte de uma situação real e pede um programa pequeno e completo.

Antes de programar, identifique as **entradas**, o **processamento** e as **saídas** do problema.

Crie uma pasta `estudos-de-caso` e um arquivo para cada caso. Execute no terminal com `node nome-do-arquivo.mjs`.

Compare o resultado do seu programa com a **saída esperada**. Se for diferente, revise o raciocínio antes de mudar o código.

Os desafios extras são opcionais, para quem terminar antes.

Mapa da lista

Caso	Situação	Conteúdo	Aula	Nível
0	Primeiro dia no estágio	Ambiente, Node.js e VS Code	1	
1	Comanda da cafeteria	Variáveis, tipos e operadores	2	
2	Situação do aluno	Condicionais `if` / `else`	3	
3	Bilheteria do cinema	Condicional `switch`	3	
4	Clima e cofrinho	Laços `for` e `while`	4	
5	Loja virtual	Funções e escopo	5	
6	Estoque e caça aos bugs	LAB: funções, arrays e depuração	7	
7	Caixa de supermercado	LAB integrador de lógica	9	
8	Boletim da turma	Modularização (mini-desafio)	8 e 10	

CASO
00

ESTUDO DE CASO

Primeiro dia no estágio

Aula 1 • Ambiente, Node.js, VS Code e o que é programação • Nível

Contexto

Você começou um estágio em uma empresa de software. No primeiro dia, a líder da equipe pede duas coisas: mostrar que você entende como funciona um algoritmo e provar que o seu computador já executa programas JavaScript.

Parte A: o algoritmo do caixa eletrônico

Os passos abaixo descrevem um saque no caixa eletrônico, mas estão fora de ordem. Numere-os na sequência correta e classifique cada um como **entrada**, **processamento** ou **saída**.

Passo	Ordem	Entrada, processamento ou saída?
A. Informar o valor do saque		
B. Retirar o cartão e as notas		
C. Inserir o cartão		
D. Verificar se há saldo suficiente		
E. Digitar a senha		
F. Liberar as notas		
G. Escolher a opção "Saque"		

E se **não houver saldo suficiente**? O que o algoritmo precisaria fazer de diferente?

Parte B: meu primeiro programa

1. Abra o terminal e confira se o Node.js está instalado com `node -v`.
2. Crie uma pasta chamada `estudos-de-caso` e abra-a no VS Code.
3. Crie o arquivo `ola.mjs`.
4. Use `console.log` para exibir um cartão de apresentação com seu nome, seu curso e seu objetivo, entre duas linhas de `=`.
5. Na última linha, exiba a versão do Node.js usando `process.version`.
6. Execute o programa com `node ola.mjs`.

Saída esperada

Terminal

```
$ node ola.mjs
================================
  Olá, mundo!
  Nome: Ana Souza
  Curso: Trilha START
  Objetivo: ser desenvolvedora
================================
Versão do Node.js: v22.22.0
```

A versão exibida pode ser diferente, de acordo com o Node.js instalado no seu computador.

Perguntas para discussão

a) Qual é a diferença entre o VS Code e o Node.js?

b) O que acontece se você executar `node ola.js`, mas o arquivo se chamar `ola.mjs`?

CASO 01

ESTUDO DE CASO

Comanda da cafeteria

Aula 2 • Variáveis, tipos primitivos e operadores • Nível

Contexto

Uma cafeteria anota os pedidos no papel e faz as contas na calculadora. O dono quer um programa que calcule a comanda automaticamente, inclusive a taxa de serviço e a divisão da conta.

Dados do pedido

Item	Preço unitário	Quantidade
Café	R$ 6,50	2
Pão de queijo	R$ 4,00	3

Cliente: **Ana** • Pessoas que vão dividir a conta: **2** • Cliente fidelidade: **sim** • Taxa de serviço: **10%**

Sua tarefa

1. Crie o arquivo `comanda.mjs` e declare uma variável ou constante para cada dado, com o tipo adequado (`string`, `number` ou `boolean`).
2. Calcule o subtotal, a taxa de serviço, o total e o valor que cada pessoa vai pagar.
3. Crie a variável `ganhaBrinde`, que deve ser `true` quando o total for de pelo menos R$ 25,00 **e** a cliente for fidelidade.
4. Use o operador `%` para descobrir se a quantidade de pães é par.
5. Exiba os valores em reais com duas casas decimais e mostre o `typeof` de três variáveis de tipos diferentes.

Saída esperada

Terminal

```
$ node comanda.mjs
Cliente: Ana
Subtotal: R$ 25.00
Taxa de serviço (10%): R$ 2.50
Total: R$ 27.50
Valor por pessoa: R$ 13.75
Ganha brinde? true
Quantidade de pães é par? false
string number boolean
```

Parte B: preveja o resultado

Sem executar, escreva o resultado de cada expressão e o tipo do resultado. Depois confira no Node.js.

Expressão	Resultado	Tipo

`'5' + 3`		
`'5' - 3`		
`10 % 3`		
`5 == '5'`		
`5 === '5'`		
`true && false`		
`!(4 > 2)`		

> **Desafio extra (opcional)**
>
> Mude `clienteFidelidade` para `false` e descubra o que muda na saída. Depois, acrescente um terceiro item ao pedido.

CASO
02

ESTUDO DE CASO

Situação do aluno

Aula 3 • Condicionais if / else if / else • Nível

Contexto

A secretaria de uma escola técnica define a situação dos alunos à mão no fim do semestre. Ela precisa de um programa que receba as três notas e a frequência e informe a situação final.

Regras da escola

Condição	Situação
Frequência menor que 75% (essa regra vale antes das demais)	Reprovado por falta
Média maior ou igual a 7	Aprovado
Média maior ou igual a 5 e menor que 7	Recuperação
Média menor que 5	Reprovado por nota
Alguma nota fora de 0 a 10, ou frequência fora de 0 a 100	Dados inválidos

Sua tarefa

1. Crie o arquivo `situacao.mjs` e leia as três notas e a frequência (use `readline/promises`).
2. Converta as respostas para número com `Number(...)`.
3. Verifique se os dados são válidos. Se não forem, exiba "Dados inválidos".
4. Calcule a média e exiba-a com uma casa decimal.
5. Use `if`, `else if` e `else` para exibir a situação, respeitando a prioridade da frequência.

Exemplo de execução

Terminal

```
$ node situacao.mjs
Nota 1: 8
Nota 2: 7
Nota 3: 9
Frequência (%): 90
Média: 8.0
Situação: Aprovado
```

Teste seu programa

Execute o programa com os dados abaixo e anote a situação que ele mostrou.

Notas	Frequência	Situação exibida
8, 7, 9	90%	

6, 5, 4	80%	
3, 4, 2	100%	
10, 10, 10	60%	
11, 5, 5	90%	

Desafio extra (opcional)

Mostre também quantos pontos faltaram para a aprovação quando o aluno ficar de recuperação.

CASO
03

ESTUDO DE CASO

Bilheteria do cinema

Aula 3 • Condicional switch • Nível

Contexto

Um cinema de bairro quer agilizar a venda de ingressos. O atendente escolhe o tipo de ingresso em um menu, informa a quantidade e o programa calcula o valor, aplicando a promoção de quarta-feira.

Tabela de preços

Opção	Tipo de ingresso	Preço
1	Inteira	R$ 40,00
2	Meia-entrada	R$ 20,00
3	Idoso (60+)	R$ 20,00
4	Cortesia	R$ 0,00

Promoção: às quartas-feiras, o valor total tem 50% de desconto.

Sua tarefa

1. Crie o arquivo `cinema.mjs` e exiba o menu com as quatro opções.
2. Leia a opção escolhida e use um `switch` para definir o preço e o nome do tipo de ingresso.
3. As opções 2 e 3 têm o mesmo preço: faça os dois `case` compartilharem o mesmo bloco.
4. Use o `default` para exibir "Opção inválida." e encerrar sem perguntar mais nada.
5. Para uma opção válida, leia a quantidade (inteiro maior que zero) e pergunte se é quarta-feira (s/n).
6. Calcule e exiba o total com duas casas decimais.

Exemplo de execução

Terminal

```
$ node cinema.mjs
1 - Inteira   (R$ 40,00)
2 - Meia      (R$ 20,00)
3 - Idoso     (R$ 20,00)
4 - Cortesia  (R$ 0,00)
Escolha o tipo de ingresso: 2
Quantidade: 3
É quarta-feira? (s/n): s
3 ingresso(s) Meia: R$ 30.00
```

Teste seu programa

Opção	Quantidade	Quarta?	Resultado exibido

1	2	n	
2	3	s	
5	–	–	
4	0	n	

Desafio extra (opcional)

Acrescente a opção 5, "Estudante com carteirinha", que custa R$ 25,00. Quantas linhas de código você precisou alterar?

CASO
04

ESTUDO DE CASO

Clima da semana e cofrinho

Aula 4 • Laços de repetição for e while • Nível

Contexto

Uma rádio local apresenta um resumo do clima da semana (Parte A). Em outro quadro do programa, os ouvintes querem saber em quanto tempo atingem uma meta de economia (Parte B).

Parte A: relatório de temperaturas (for)

Dom	Seg	Ter	Qua	Qui	Sex	Sáb
28°C	31°C	33°C	29°C	35°C	30°C	27°C

1. Crie o arquivo `temperaturas.mjs` com dois arrays: um com os dias e outro com as temperaturas.
2. Use um laço `for` para exibir cada dia com sua temperatura.
3. No mesmo laço, calcule a soma, a maior e a menor temperatura, o dia mais quente e quantos dias passaram de 30°C.
4. Depois do laço, exiba a média (uma casa decimal) e os demais resultados.

Terminal

```
$ node temperaturas.mjs
Dom: 28°C
...
Sáb: 27°C

Média da semana: 30.4°C
Maior: 35°C (Qui)
Menor: 27°C
Dias acima de 30°C: 3
```

Parte B: cofrinho (while)

Uma pessoa quer juntar **R$ 1.000,00**. Todo mês o saldo rende **1%** e, depois do rendimento, ela deposita **R$ 150,00**. O saldo começa em zero.

1. Crie o arquivo `cofrinho.mjs` com constantes para a meta, o depósito e o rendimento.
2. Use um laço `while` que se repete enquanto o saldo for menor que a meta.
3. A cada mês: aplique o rendimento, some o depósito e exiba o saldo.
4. Ao final, informe em quantos meses a meta foi atingida.

Terminal

```
$ node cofrinho.mjs
Mês 1: R$ 150.00
Mês 2: R$ 301.50
...
Mês 7: R$ 1082.03
Meta atingida em 7 meses!
```

Para discutir

Por que a Parte A usa `for` e a Parte B usa `while`?

> **Desafio extra (opcional)**
>
> Na Parte B, peça a meta e o depósito ao usuário. O que acontece se o depósito for zero? Como evitar esse problema?

CASO
05

ESTUDO DE CASO

Loja virtual

Aula 5 • Funções simples e escopo • Nível

Contexto

Uma loja virtual repete os mesmos cálculos em várias partes do site: subtotal, desconto, frete. A equipe decidiu organizar cada cálculo em uma função que possa ser reaproveitada.

Parte A: funções da loja

Crie o arquivo `loja.mjs` com as funções abaixo. Todas devem receber parâmetros e devolver o resultado com `return`, exceto `resumirPedido`, que exibe o resumo.

Função	O que faz
`calcularSubtotal(preco, quantidade)`	Retorna preço × quantidade.
`aplicarDesconto(valor, percentual = 0)`	Retorna o valor com o desconto aplicado. Se o percentual não for informado, vale 0.
`calcularFrete(valor)`	Retorna 0 (frete grátis) se o valor for de pelo menos R$ 200,00. Caso contrário, retorna 25.
`formatarMoeda(valor)`	Retorna um texto como "R$ 216.00".
`resumirPedido(produto, preco, quantidade, desconto)`	Usa as funções anteriores e exibe subtotal, valor com desconto, frete e total.

Teste com dois pedidos: **Fone de ouvido**, R$ 120,00, 2 unidades, 10% de desconto; e **Mouse**, R$ 80,00, 1 unidade, sem informar desconto.

Terminal

```
$ node loja.mjs
--- Fone de ouvido ---
Subtotal: R$ 240.00
Com desconto: R$ 216.00
Frete: R$ 0.00
Total: R$ 216.00
--- Mouse ---
Subtotal: R$ 80.00
Com desconto: R$ 80.00
Frete: R$ 25.00
Total: R$ 105.00
```

Parte B: escopo, preveja a saída

Leia o código abaixo **sem executar** e escreva o que cada `console.log` exibe.

`escopo.mjs` Não execute antes de responder

```
let mensagem = 'global';

function testarEscopo() {
  let mensagem = 'função';
  if (true) {
    let mensagem = 'bloco';
    console.log('1:', mensagem);
  }
  console.log('2:', mensagem);
}

testarEscopo();
console.log('3:', mensagem);

function contar() {
  let contador = 0;
  contador++;
  return contador;
}

console.log('4:', contar());
console.log('5:', contar());
console.log('6:', contador);
```

Linha exibida	Sua resposta	Por quê?
1:		
2:		
3:		
4:		
5:		
6:		

CASO
06

ESTUDO DE CASO

Controle de estoque e caça aos bugs

Aula 7 • LAB: funções, arrays e depuração • Nível

Contexto

Uma loja de informática controla o estoque em uma planilha e quer um programa que responda às perguntas do gerente. Enquanto isso, um colega pediu ajuda com um programa de vendas que "roda, mas dá resultado errado".

Produtos cadastrados

Nome	Preço	Estoque
Teclado	R$ 150,00	10
Mouse	R$ 80,00	0
Monitor	R$ 900,00	4
Webcam	R$ 200,00	5

Parte A: sua tarefa

1. Crie o arquivo `estoque.mjs` com três arrays paralelos: `nomes`, `precos` e `estoques`. A mesma posição representa o mesmo produto.
2. Crie `calcularValorEstoque(precos, estoques)`, que retorna a soma de preço × quantidade de todos os produtos.
3. Crie `listarSemEstoque(nomes, estoques)`, que retorna um array com os nomes dos produtos com estoque zerado.
4. Crie `encontrarMaisCaro(precos)`, que retorna a **posição** do produto mais caro.
5. Crie `calcularReajuste(preco, percentual)` e aplique 10% ao preço do Teclado.
6. No programa principal, chame as funções e exiba os resultados. Todas as funções devem usar `return`.

Saída esperada

Terminal

```
$ node estoque.mjs
Valor total em estoque: R$ 6100.00
Sem estoque: Mouse
Mais caro: Monitor (R$ 900.00)
Teclado com reajuste de 10%: R$ 165.00
```

Desafio extra (opcional)

Crie `listarEstoqueBaixo(nomes, estoques, limite)`, que retorna os produtos com estoque menor que o limite. Com limite 5, quais aparecem?

Parte B: caça aos bugs

O programa abaixo deveria mostrar a média de vendas (90), dizer se a meta de 100 foi atingida e somar um bônus de 50 à média. Ele roda sem travar, mas exibe resultados errados. Encontre os **quatro** erros e explique cada um.

bugado.mjs Contém 4 erros de lógica

```
// Calcula a média de vendas da semana e verifica a meta
const vendas = [120, 80, 70];
const META = 100;

function calcularMedia(lista) {
  let soma = 0;
  for (let i = 0; i <= lista.length; i++) {
    soma = soma + lista[i];
  }
  soma / lista.length;
}

let media = calcularMedia(vendas);
console.log('Média: ' + media);

if (media = META) {
  console.log('Meta atingida!');
} else {
  console.log('Meta não atingida.');
}

const bonus = '50';
console.log('Média com bônus: ' + (media + bonus));
```

Saída atual (errada)

```
$ node bugado.mjs
Média: undefined
Meta atingida!
Média com bônus: 10050
```

Nº	Trecho com erro	Explicação e correção
1		
2		
3		
4		

CASO
07

ESTUDO DE CASO

Caixa de supermercado

Aula 9 • LAB integrador de lógica • Nível

Contexto

Um mercadinho de bairro quer trocar o caderno do caixa por um programa simples. O operador registra os produtos um a um, encerra com "fim" e o programa emite um cupom e aplica o desconto da forma de pagamento.

Requisitos

1. Crie o arquivo `caixa.mjs`. Em um laço, peça o nome do produto até o operador digitar `fim` (em maiúsculas ou minúsculas).
2. Para cada produto, leia o preço unitário (número maior que zero, aceitando vírgula) e a quantidade (inteiro maior que zero).
3. Guarde cada item como um objeto `{ nome, preco, quantidade }` em um array.
4. Se nenhum item for registrado, exiba "Nenhum item registrado." e encerre.
5. Exiba o cupom: uma linha por item com o total do item e, no fim, o subtotal.
6. Leia a forma de pagamento e use `switch` para definir o desconto. Repita a pergunta se a opção for inválida.
7. Exiba o desconto e o total a pagar.

Formas de pagamento

Opção	Forma	Desconto
1	Dinheiro	5%
2	Pix	3%
3	Cartão	Sem desconto

Regras de validação

Nome do produto vazio deve ser recusado.

Preço e quantidade inválidos devem ser pedidos de novo, sem perder os itens já registrados.

Crie ao menos uma função auxiliar, por exemplo `lerNumero(pergunta)` e `formatarMoeda(valor)`.

Exemplo de execução

Terminal

```
$ node caixa.mjs
Produto (ou "fim"): Arroz
Preço unitário: 25,90
Quantidade: 2
Produto (ou "fim"): Feijão
Preço unitário: 8.50
Quantidade: 3
Produto (ou "fim"): Café
Preço unitário: 18
Quantidade: 1
Produto (ou "fim"): fim

===== CUPOM =====
2x Arroz: R$ 51.80
3x Feijão: R$ 25.50
1x Café: R$ 18.00
Subtotal: R$ 95.30
Pagamento: 1-Dinheiro 2-Pix 3-Cartão: 2
Desconto (3%): R$ 2.86
Total a pagar: R$ 92.44
```

> **Desafio extra (opcional)**
>
> Mostre também qual foi o item mais caro do cupom (maior total) e quantas unidades foram vendidas no total.

CASO
08

ESTUDO DE CASO

Boletim da turma em módulos

Aulas 8 e 10 • Modularização, revisão e mini-desafio • Nível

Contexto

A coordenação gostou do programa de situação do aluno (Caso 2) e agora quer um boletim da turma inteira. Como o sistema vai crescer, o código precisa ser dividido em módulos com responsabilidades separadas.

Organização dos arquivos

Arquivo	Responsabilidade
`entrada.mjs`	Exportar `lerTexto(pergunta)`, `lerNumero(pergunta, minimo, maximo, inteiro)` e `fecharEntrada()`. Toda a leitura e a validação ficam aqui.
`notas.mjs`	Exportar `calcularMedia(notas)`, `definirSituacao(media)` e `encontrarMelhorAluno(alunos)`. Só cálculos: sem perguntas e sem `console.log`.
`turma.mjs`	Programa principal: importar as funções, coletar os dados e exibir o relatório.

Requisitos

1. Pergunte quantos alunos há na turma (inteiro de 1 a 50).
2. Para cada aluno, leia o nome (obrigatório) e três notas (de 0 a 10, aceitando vírgula).
3. Guarde cada aluno como `{ nome, media, situacao }`. Situação: Aprovado (média ≥ 7), Recuperação (média ≥ 5) ou Reprovado.
4. Exiba o relatório: uma linha por aluno, a média da turma, o aluno de melhor desempenho e quantos foram aprovados.
5. Comente as principais etapas do código.

Exemplo de execução

Terminal

```
$ node turma.mjs
Quantos alunos? 3
--- Aluno 1 ---
Nome: Ana
Nota 1: 8
Nota 2: 9
Nota 3: 10
--- Aluno 2 ---
Nome: Bruno
Nota 1: 6
Nota 2: 5
Nota 3: 5,5
--- Aluno 3 ---
Nome: Carla
Nota 1: 4
Nota 2: 3
Nota 3: 5

===== RELATÓRIO DA TURMA =====
Ana: 9.0 - Aprovado
Bruno: 5.5 - Recuperação
Carla: 4.0 - Reprovado
Média da turma: 6.2
Melhor desempenho: Ana (9.0)
Aprovados: 1 de 3
```

Planejamento da solução

Antes de programar, descreva como os dados vão circular entre os três arquivos.

Desafio extra (opcional)

Reaproveite `notas.mjs` em outro programa, sem alterar nada nele, para calcular a média de um único aluno. O que isso mostra sobre as vantagens da modularização?